Ejercicio de evaluación continua 1

Programación Declarativa Aplicada
Máster en Ingeniería Informática
Facultad de Informática - UCM

- **Duración:** 1 hora
- Pasada la hora límite, se pueden seguir entregando ejercicios hasta el día 26 de octubre (incluido). Sin embargo, solamente se obtendrá como máximo el 25 % de la calificación obtenible en los mismos. La segunda entrega se realizará a través del Campus Virtual.
- No se permite el uso de herramientas de IAGen en la elaboración de los ejercicios.

1. *[2 puntos]* Supongamos una lista de pares $\{clave, valor\}$. Implementa una función recursiva llamada `keydelete/2` que, a partir de una lista de pares y una clave `k`, devuelva esa misma lista pero eliminando el par cuya primera componente tenga el valor `k`. Si hay varios pares que tengan `k` como primera componente, solamente debe omitirse el primero de ellos. Por ejemplo:

```
> xs = [{4, "Abril"}, {2, "Febrero"}, {3, "Marzo"}, {3, "Miércoles"}]
> Ejercicio1.keydelete(xs, 2)
[{4, "Abril"}, {3, "Marzo"}, {3, "Miércoles"}]
> Ejercicio1.keydelete(xs, 3)
[{4, "Abril"}, {2, "Febrero"}, {3, "Miércoles"}]
```

2. *[3 puntos]* La función `with_index/1` recibe una lista $xs = [x_0, x_1, \ldots, x_{n-1}]$ y devuelve una lista de pares $[\{x_0, 0\}, \{x_1, 1\}, \ldots, \{x_{n-1}, n-1\}]$, donde cada elemento de la lista original está emparejado con su posición dentro de la lista. Suponemos que empezamos a numerar los elementos desde el 0.

```
> Ejercicio1.with_index(["a", "b", "c"])
[{"a", 0}, {"b", 1}, {"c", 2}]
```

   a) *[1.5 puntos]* Implementa `with_index/1` de manera recursiva. Puedes definir las funciones auxiliares que necesites.

   b) *[1.5 puntos]* Utilizando `with_index/1` y `Enum.reduce/3`, escribe una función `last_pos(xs, x)` que, dada una lista `xs` y un elemento `x`, devuelva la posición de la última aparición de `x` en `xs`. Si `x` no aparece en `xs`, la función debe devolver `nil`.

```
> Ejercicio1.last_pos([:a, :b, :c, :b], :c)
2
> Ejercicio1.last_pos([:a, :b, :c, :b], :b)
3
> Ejercicio1.last_pos([:a, :b, :c, :b], :e)
nil
```

3. *[2 puntos]* Dada una lista de números enteros, decimos que un elemento es un *hermano mayor* si su valor es estrictamente mayor que el valor situado a su izquierda en la lista. Por ejemplo, en la lista `[5, 7, 3, 3, 9, 10]` hay tres hermanos mayores: el 7, el 9 y el 10. Utilizando `Enum.zip/2` y `Enum.count/2`, escribe una función `num_hermanos_mayores` que devuelva el número de hermanos mayores que contiene la lista pasada como parámetro.

```
> Ejercicio1.num_hermanos_mayores([5, 7, 3, 3, 9, 10])
3
> Ejercicio1.num_hermanos_mayores([10, 7, 3])
0
> Ejercicio1.num_hermanos_mayores([6])
0
```

4. *[2 puntos]* Suponemos que queremos representar la información de una persona mediante una tupla de tres componentes: `{Nombre, Edad, Sueldo}`. Por ejemplo, `{"Laura", 25, 1900}`. Suponemos que tenemos una lista de tuplas como la anterior. Por ejemplo:

   ```
   ps = [{"Laura", 25, 1900}, {"David", 15, 0}, {"Fernanda", 20, 1500}, {"Joaquín", 29, 2500}]
   ```

   Utilizando solamente funciones de orden superior (es decir, sin recursión explícita):

   a) *[1 punto]* Escribe una función que, dada una lista de tuplas como la anterior, devuelva los nombres de las personas con 18 años o más.

   ```
   > Ejercicio1.mayores_de_edad(ps)
   ["Laura", "Fernanda", "Joaquín"]
   ```

   b) *[1 punto]* Escribe una función que, dada una lista de tuplas como la anterior, devuelva la suma de los sueldos de aquellas personas con 21 años o más.

   ```
   > Ejercicio1.suma_sueldos(ps)
   4400
   ```

5. *[2 puntos]* La función `iterate/3` recibe una función `f` de aridad 1, un valor inicial `i` y un número positivo `n`, y devuelve el resultado de aplicar reiteradamente `n` veces la función `f` al valor inicial `i`. Por ejemplo, si definimos la función `incrementar/1` del siguiente modo:

   ```
   def incrementar(x), do: x + 1
   ```

   entonces tenemos:

   ```
   > Ejercicio1.iterate(&incrementar/1, 1, 3)
   4

   > Ejercicio1.iterate(&incrementar/1, 20, 5)
   25
   ```

   a) *[1 punto]* Implementa la función `iterate` de manera recursiva.

   b) *[1 punto]* Utilizando `iterate`, escribe una función `pow2(x)` que calcule 2^x.